

CS 233 OBJECT ORIENTED PROGRAMMING AND DESIGN PATTERNS (OOPDP)



Shamama Anwar

Assistant Professor

Department of Computer Science and Engineering

Birla Institute of Technology, Mesra.

shamama@bitmesra.ac.in

9431103880

Module - III

Object Oriented Concepts: Polymorphism & Inheritance

- Controlling access to class members
- the use of this keyword
- getters and setters
- Composition, enum, the use of static and final
- Garbage collection
- Inheritance
 - Superclass and subclass, protected members, constructors in subclass, the Object class,
- Introduction to polymorphism
 - Abstract classes and methods, Assignment between subclass and superclass variables, Creating and using interfaces.

Constructors

- Initialize an object of a class
- Java requires a constructor for every class
- Java will provide a default no-argument constructor if none is provided
- Called when keyword new is followed by the class name and parentheses
- They have the same name as that of the class.
- Constructors can be overloaded

Constructor vs Methods

Constructor	Method
No return type	Can have return type
Same name as that of the class	Any name (valid identifier)
Invoked as soon as object is created	Invoked explicitly by using the object / class name
Not inherited	Can be inherited
Generally used to initialize objects	Generally used to perform a task
Cannot be abstract, final, static or synchronized	Can use any of the modifiers
Can be overloaded	Can be overloaded

Example

```
class Circle
{
    public double r;
    Circle()
    { r = 3; }
    Circle(double a) ← Parametrized Constructor
    { r = a; }
    public double circumference()
    { return 2*3.14*r; }
    public double area()
    { return 3.14 * r * r; }}

public class demo
{
    public static void main(String args[])
    {
        Circle c1 = new Circle();
        Circle c2 = new Circle(2.6);
        System.out.println("C1:Area"+c1.area()+
            "Circumference"+c1.circumference());
        System.out.println("C2:Area"+c2.area()+
            "Circumference"+c2.circumference());
    }
}
```

this keyword

- *this* keyword can be used to refer to the object itself.
- It is generally used for accessing class members (from its own methods) when they have the same name as those passed as arguments.

```
class Circle
{
    public double r;
    Circle()
    { r = 3; }
    Circle(double r)
    { this.r = r; }
    public double circumference()
    { return 2*3.14*r; }
    public double area()
    { return 3.14 * r * r; }}
```

Static Members

- Java supports definition of global methods and variables that can be accessed without creating objects of a class. Such members are called Static members.
- Define a variable by marking it with the `static` keyword.
- This feature is useful when we want to create a variable common to all instances of a class.
- One of the most common example is to have a variable that could keep a count of how many objects of a class have been created.
- Java creates only one copy for a static variable which can be used even if the class is never instantiated.

Static Members

```
class Circle
{
    public double r;
    public static int numCircles = 0;
    Circle() { r = 3; numCircles++; }
    Circle(double a) { r = a; numCircles++; }
    public double circumference() { return 2*3.14*r; }
    public double area() { return 3.14*r*r; }
}

public class demo
{
    public static void main(String args[])
    {
        Circle c1 = new Circle();
        System.out.println("Count:"+Circle.numCircles);
        Circle c2 = new Circle(2.6);
        System.out.println("Count:"+Circle.numCircles);
    }
}
```


Static Members

```
class test1
{ int x = 0;
  static int y = 0;
  void setX(int n) { x = n; }
  void setY(int n) { y = n; }
  int getX() { return x; }
  int getY() { return y; }
}

class test
{ public static void main(String args[])
  { test1 t1 = new test1();
    test1 t2 = new test2();
    t1.setX(9);    t2.setX(5);
    System.out.println("T1:"+t1.getX());
    System.out.println("T2:"+t2.getX());
    t1.setY(12);
    System.out.println("T1:"+t1.getY());
    System.out.println("T2:"+t2.getY());
    t2.setY(56);
    System.out.println("T1:"+t1.getY());
    System.out.println("T2:"+t2.getY());    } }
```

Static Methods

- A class can have methods that are defined as static (e.g., main method).
- Static methods can be accessed without using objects i.e. there is NO need to create objects.
- They are prefixed with keyword “static”
- Static methods are generally used to group related library functions that don't depend on data members of its class. For example, Math library functions.
- **Difference between local variables, Instance variables and Class (Static) variables ??**

Final instance variable

- The keyword final is used to declare a constant.
- It can be used with variables as well as argument declaration.

```
class test
{
    final int MAX = 10;
    void show()
    {
        System.out.println(MAX);
        MAX++;
        System.out.println(MAX);
    }
}

public class demo
{
    public static void main(String args[])
    {
        test t1 = new test();
        t1.show();
    }
}
```

Final instance variable

```
class test
{
    final int MAX = 10;
    void show(final int x)
    {
        System.out.println(MAX);
        x++;
        System.out.println(x);
    }
}

public class demo
{
    public static void main(String args[])
    {
        test t1 = new test();
        t1.show(12);
    }
}
```

Programs

- WAP to find the area of a square, rectangle and circle. Write appropriate constructors and methods.
- WAP to add two complex numbers.

Garbage collection

- It is a Memory Management technique that frees the memory locations that are no longer referenced by the program.
- Combat heap fragmentation.
- Relieves programmer from manual freeing the memory.
- Helps in ensuring program integrity.
- Disadvantages
 - Extra Overhead.
 - Less control over scheduling of CPU time.

Garbage collection

- **Purpose**

- Find and delete unreachable objects.
- Free space as much as possible.

- **When??**

- GC is under control of JVM.
- One can request but no guarantees.

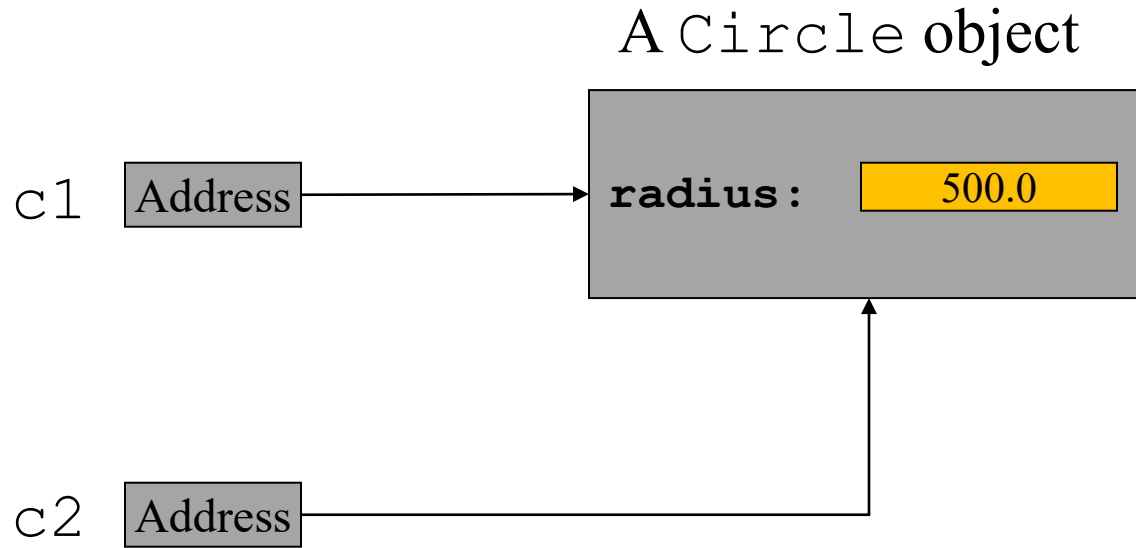
- **How??**

- Discovery of unreachable objects.
- With the help of Algorithms.

Garbage collection

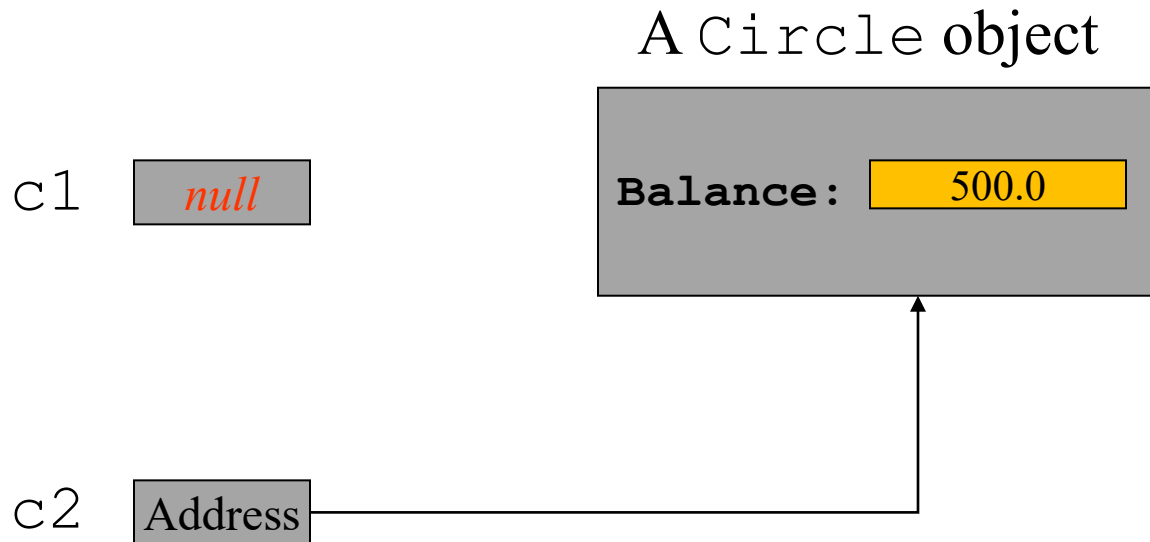
- **Ways for making objects eligible for collection**
 - Nulling a reference
 - Reassigning a reference variable
 - Isolating a reference
- **Forcing garbage collection**
 - Methods available to perform GC
 - Only requests and no demands
 - Using static method `System.gc()`

Garbage collection



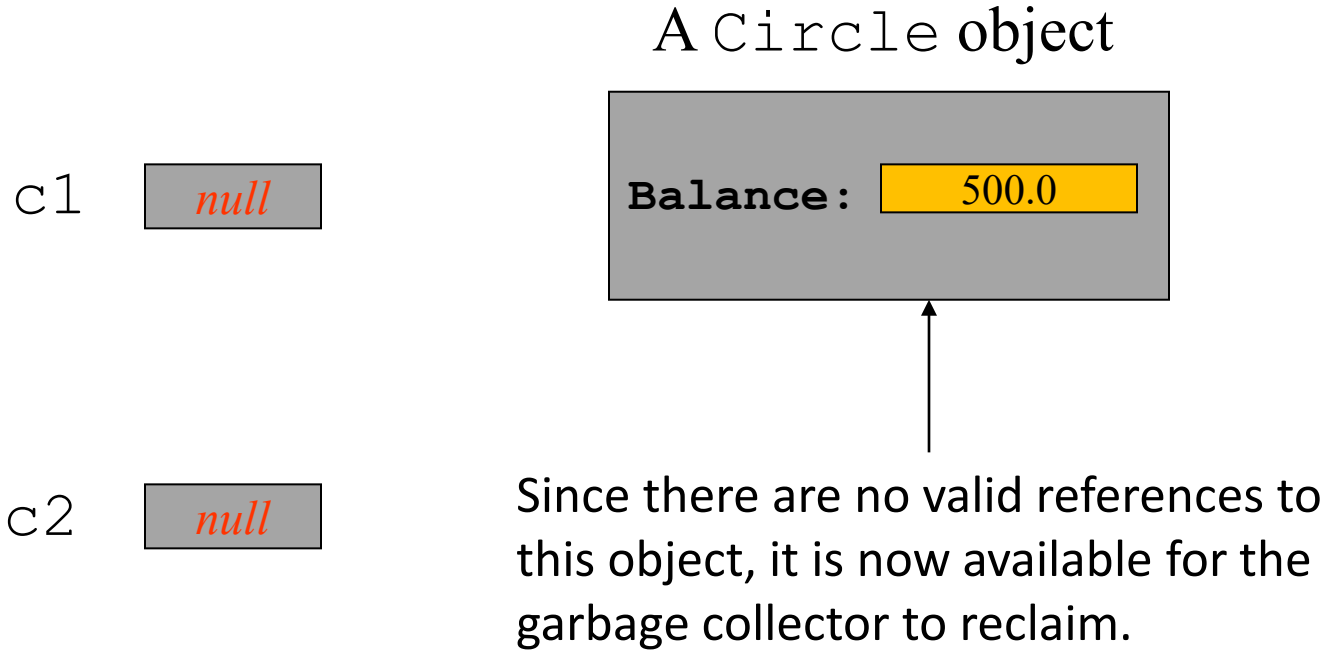
Here, both `c1` and `c2` point to the same instance of the `Circle` class.

Garbage collection



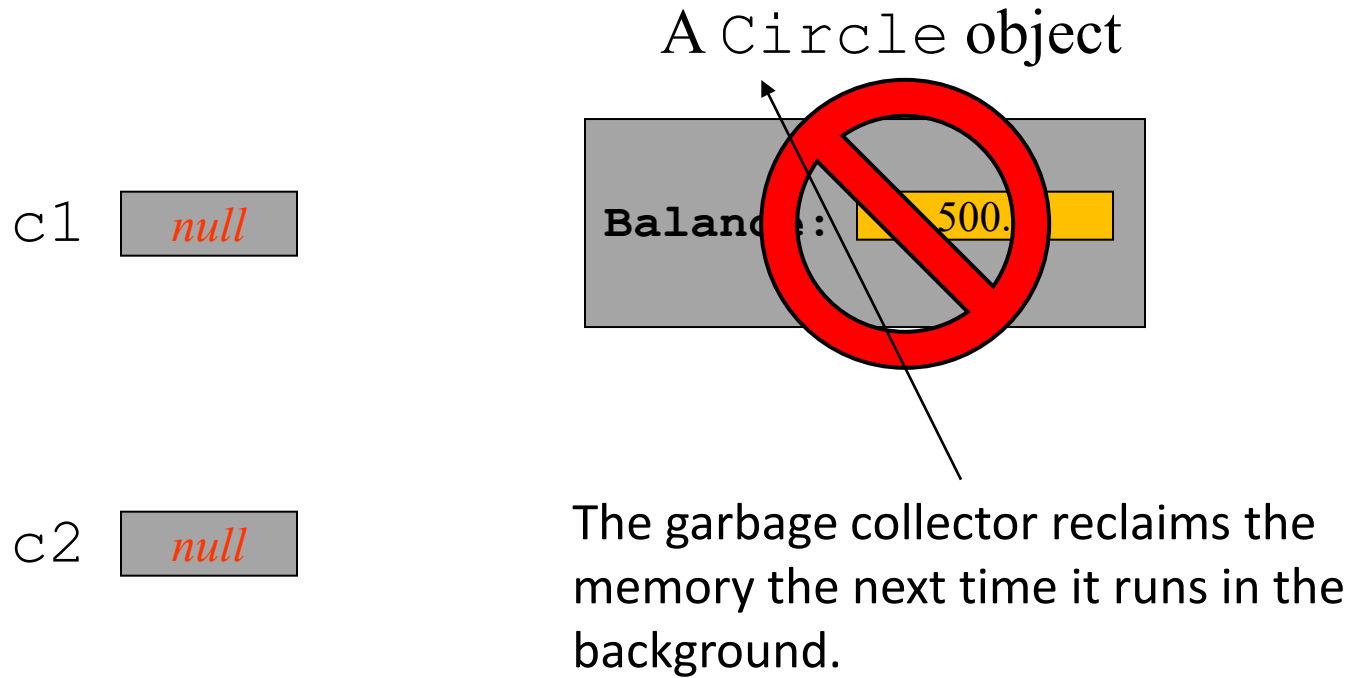
However, by running the statement: **`c1 = null;`** only `c2` will be pointing to the object.

Garbage collection



If we now run the statement: **c2 = null;** neither c1 or c2 will be pointing to the object.

Garbage collection



Garbage collection approaches

1. Reference Counting

- Maintains a reference count for each object.
- Count increases as number of reference of a particular object increases.
- As the referencing object move to other objects, the reference count is decremented.
- Eligible for GC when count equals zero.

Advantages

- Can run in small chunks of time
- Suitable for real time environments

Disadvantages

- Overhead of incrementing and decrementing reference count.

Garbage collection approaches

2. Tracing

- Traces the entire set of objects.
- All objects having reference are marked.
- Memory is reclaimed for objects that remain unmarked.
- Also known as Mark and Sweep algorithm.
- It uses compaction and Copying for fragmentation problem.

Disadvantages

- Requires jumping all over memory.

finalize() method

- Since objects can hold resources (ie. open file, connection to database, etc), it is important to release those resources when an object is destroyed.
- When the garbage collector is ready to return an object's memory to the system, it invokes the object's `finalize()` method.
- Remember, the programmer has no control over the garbage collector. As a result, the programmer has no control over when this method will be invoked (if ever).
- The `finalize()` method is written as: **`public void finalize(){...}`**
If it is included in a class, it will run just prior to the garbage collector reclaiming its memory.
- The garbage collector is a background thread that runs periodically.
- It cannot be determined when the `finalize` method will actually be run.

finalize() method

```
public class TestGarbage1
{
    public void finalize()
    {
        System.out.println("object is garbage collected");
    }
    public static void main(String args[])
    {
        TestGarbage1 s1=new TestGarbage1();
        TestGarbage1 s2=new TestGarbage1();
        s1=null;
        s2=null;
        System.gc();
    }
}
```